

We are only scratching the surface of what generative AI can do. Chat is just the first, simplest and most overused AI interface. While everyone's stuck in the chat box, let's step outside and unlock a new way to create value with agents. With one command, adz I've enabled a suite of agentic workflows. All we have to do is drag and drop the right files into the right directories to kick off one of eight specialized agentic workflows. If we drag and drop cast. Txt into the generate images zone, we're going to kick off an agentic workflow that uses the new Nano Banana aka Gemini 2.5 flash model. This is going to generate images at scale based on whatever was in our prompt. Let's edit some images as well with our edit image drop zone. Let's process our monthly finances for August Finance Drop zone. I have this video that I want to transcribe and format. So we'll drag and drop this into the morning debrief Twitter classification data set that I need to expand training data set Drop zone drop after drop we're activating unique agentic workflows that are going to operate end to end, accomplishing work for us with a single drop of a file. Let's get out of the chat interface and talk about agentic drop zones. A lot of engineering work is file based. What does that mean exactly? The architecture for this workflow looks like this. We have input files that we drop into specific directories. These directories are programmed to kick off specific agents that run specific prompts that produce specific outcomes. We have this unique drops YAML file that that configures our entire system. The great part about this is that it's agent agnostic. You can fire up any prompt you want and then it gets the work done. We're in the age of agents and with agentic drop zones we have a new system to automate file based agentic workflows with ease. And it's all based on this one drops YAML file, this one configuration file that's insanely simple to operate. Now let's see how our workflows are doing. You can see here we have some images generated. Let's check them out. This is using the new Google Nano Banana and image generation model. As you can see here, we just have three great photos of cats. Our Twitter classification CSV file got additional rows added to that. So we'll take a look at that in a second. And we have our edit cat images here. This workflow is saving all the Prompts side by side, the images that were generated. Fantastic. This was our Create image workflow. We drag and dropped over the generate images directory. Now we have edit images. So nanobanana is famous for, for its ability to edit images. Check this out. So here's the original image and here's a version with the fur of the cat gray. Now, you know this is pretty incredible, right? Very accurate. It's maintaining a lot of the detail. And we have a version of a black cat here as well. Black cat, gray cat, yellow cat. The image edits are nearly perfect. And here in our output of this agentic workflow, you can see the exact prompts we use. There's a source, there's the replicate URL. We're going through the replicate MCP server and replicate API 2 workflows right there. Now we

also had our, our Twitter classification typical data classification problem where you just have a CSV of several columns. In this case we're just looking at example mock tweets. And what we wanted this workflow to do for us was extend this data set. So you can imagine you have a data set that you want to extend with additional information. And if we move to the left here, you can see we generated 25 new rows based on the existing rows. That's the key here. It created that extended dataset, final output and a report for us. We have agentic workflows getting work done for us in the background. They're specialized workflows we are getting out of the chat interface. All right, so you can see we have our finance categorizer. It's finishing up some work here. And our other agent workflows have summarized their work already. You can see exactly how powerful this can be. And here we go. So here's our finances. Our agent generated several assets for us. You can do anything under the sun with agentic workflows. All right, there we go. It's got a summary there. You can see in this mock data set, we're going crazy on our AI services. Are Amazon bills crazy? Let's go ahead and look at some of the generated assets. Right, so here's spending by category. This workflow looked at our finances and it generated a concrete pie chart for us, breaking it all down. 55k on AI services this month. We might have an AI addiction. Who knows? Amazon, of course, Cloud technology, Compute. And then it goes down from there. Right? Here's the original input CSV file that our agent looked at. And here is the categorized input statement that our agent looked at. So it's Taking all these rows, categorizing them and then operating on them. Right. So we also got this nice graph here showcasing a couple key expenses and it has the start and the end of the month. This is just a random example that I'm showcasing here. Right. So we also just had our transcription finished. Right? So remember we dropped that audio file into morning debrief zone and so this is what it came up with here, Right? If we open this up, you can see five agent interaction patterns for engineering impact. This was the transcript for last week's video. Top three priorities, some key ideas, Human in the loop, trade offs, five level scalability framework, ad hoc prompts, reusable prompts, sub agents, MCP servers, full apps. We're in the age of agents, not LLMs. Agents provide exponential leverage. Great stuff. It's creating some extensions. So our agent is extending the original ideas from whatever transcript we drop in. It's got some interesting questions for us. And then here we have the full on transcript. We're using OpenAI whisper with the tiny model. Here's the opener for last week's video. If you're an engineer using the best tool for the job, you know this as a fact. Agents are not the future. They're already on our desks, blah blah blah, so on and so forth. Check out last week's video to see the five agent interaction patterns for engineering impact. We have all types of engineering work getting accomplished for us just based on dropping in the right file into the right zone. What's going on here? How were we able to specialize so many different workflows and set up this cool drop zone pattern? The first thing I want to highlight here, we have the entire application packed into an sfs, a single file script. Okay, so this one Python file using Astral UV is loading dependencies right inside the file and

doing all of this work for us in some 700 lines of code. I love when I can pack in a ton of value to a single script. So how does this work? This all operates with a set of prompts. You can see we have dot clot here. It operates with our single file script and most importantly our drops YAML file. Let's break down how the drops work and how you can configure brand new drop zones. And as we're working through this, think through what types of workflows that are file based that you could use this system for. Think about repeat workflows where you're just taking a file and you need to operate on it. All right, so this is what A drop zone configuration entry looks like. We of course have the name of the drop zone. We have our file patterns. This drop zone will only kick off if we drop in a text file. We have the prompt that we want to kick off. And so we can go ahead and just open up Echo MD and you can see what this prompt looks like. Exactly. We have our purpose, we have our variables, we have the workflow, and then we have our example output format here. And now we can kick this workflow off right now, right? This is the simplest possible working version of the application. Here's Echo Txt. Let's go ahead and copy this so we can run this twice. Let's drag one into echo zone. Right? So we're going to kick off that simple echo prompt and let's drag one into. I have a Gemini echo zone here as well. This is agent agnostic, so this works with any agent you want. Obviously cloud code is going to be the most powerful agent. But I also have this workflow working with Gemini. We can drag and drop that in here. Cloud code Echo drop sound workflow. I'll execute the echo command workflow for the drop file. Simple Gemini CLI drop zone test. And this is just the content that was in that file, right? And if we go back over to this, you can see that's exactly what we wanted the output to be. If we open up, if we control Z that file and just take a look at that. You can see in this file we just have simple Gemini CLI agentic drop zone test. And it just echoed that for us. Okay, so you can see how this works end to end with a simple example like this, right? We have a drop zone configured. This triggers a prompt. We're using a special file path string to make this prompt agent agnostic. If we were using cloud code's arguments like this, we would be locked into cloud code. And then we're specifying a, you know, a couple of variables here that get used throughout the actual workflow definition of this prompt. And as you create your agentic workflows, this is a great prompt structure to use, right? Very concise, very simple. You have the purpose at the top, variables come next. LLMs agents are good enough to reference your variables throughout the rest of the prompt. So you can see here we're doing that Exactly. And then the workflow step of course, breaks down step by step what your agent should do. And then at the bottom you can always specify some type of reporting header structure with the details of what you want the output to look like for this agentic workflow, so that's the echo md. You can see we have specific events, so we have event types for the file. Created, modified, deleted, moved on, created or modified. We're going to kick off this drop zone agentic workflow. And you saw that right here. Looks like Gemini had an error here. Too many requests. Okay, whatever. You can see that when we drop this in here, right? Let's go ahead and copy this again. When we drag this in to our

echo zone, this is going to count as a file creation from the perspective of this directory, right? So we have a watcher looking at this directory and it's going to kick off this respective agent workflow. This is how this works at a high level, right? We can of course specify the agent, we can specify the model and then the color that's used throughout the logging process. And then we have one more additional kind of configuration variable here. This single drops file controls everything we need to get out of the chat interface and use a different file input based AI interaction layer. So this is interesting, right? Don't underestimate how powerful this can be. You can put anything you want in one of these prompts and you can pass in any file type, any file configuration that your prompt then reads and supports. Okay, so for example, what do I mean by that exactly? If we look at the image generation drop zone, as you know, this is going to kick off image generation. So same format, right? Reading in all text and markdown files. Then we'll kick off create image markdown. So let's go ahead and open this up, check this out, right? So we have a typical prompt format, all of our sections, so we can just collapse. And we can see we have create image variables and workflow. So this is a simple agentic prompt with these three key sections, several variables here, Drop file path. This will get replaced with whatever file we dropped in. Okay, we have our image output directory and then we have an image model variable here. You can see we're using the replicate API model syntax for this. And then we're running a workflow. First you read the file, create the output directory and then for every image detailed in this file here, do the following work. And so if we go into our generate images zone and go to the archived files and just pull back up cats. You can see here, you know we just had three simple prompts. But we can update this file to really do and be anything, right? So drag and drop this file here. We can do something simple like this, right? We'll just go ahead and use CURSOR add seven more image generation prompts. And so this is where this becomes really, really powerful. We can use agents on files. We can use LLMs to generate the input files at scale. We can detail whatever work we want done, whatever work that are drop zones, except as files, right? So we have this enhanced CATS txt and let's go ahead. You know, we can take a look at it again here. That looks great. Okay. Gemini is freaking out. I actually have no idea what's going on there. So we're going to reset our drop zone there. Now we have this new CATS file, right? So let's say CATS enhanced. Let's go ahead and drop this in, right? I'll go ahead and copy it so we can reuse it. And then I'll drag and drop it into our image generation zone. This is going to kick off that agentic workflow. You can see all the configuration right here. If we open up that detailed prompt again, it's just reading that file, right? So it's going to read the file and then it's just going to do this operation based on the contents in the file, right? So important, for every image prompt, detailed and dropped file path, you can see we're referencing that variable. It's very important. This is an advanced capability of modern LLMs. You can reference variables and set them to arbitrary things inside your prompts. So we have that and then we have our image loop. So for each image, do this work, okay? And then we're calling the

replicate MCP server and doing some arbitrary work, right? You get the idea. After all images are generated, open the directory, right? So wherever they were generated, I'm saying open that directory and then move the drop file path into the archive. And so our agent right now is doing all that work for us. It looks like I just ran out of replicate credit, so give me one second. All right, so you can see this workflow kicking off and getting work done. So the first image is ready. Now. It's going to run just the steps inside the workflow, right? It's just running top to bottom, whatever we define inside the prompt. Okay. And this is, this is the value of a system like this, right? The reusability is off the charts. Keep in mind, whatever idea you're having, right? Whatever input based agentic workflow you want to build, you just, just add an entry like this, you define the prompt, and that's it. It's quite simple, right? This is something that I've been using for quite some time now, and it's given me A crap ton of value. It's so easy to set up. Just create a text file. Create whatever file you have, whether text entries, whether you're editing files. See here we've enabled text MD and JSON for whatever image edits you want to make. We also have a training data generation. So let's say you're working with CSV or let's say you're fine tuning a model, right? So you have that.JSON I. You can just generate more training data based on whatever prompt you have. Any system that's simple, that's extensible, that's interoperable with existing technologies is going to give you a massive advantage, right? Especially when it's letting you access more compute. We're using the oldest interface known to engineers, right? Right outside the terminal, we have the operating system, right? We have the finder, we have the Explorer. It's folders and files. All the great ideas I think are obvious in retrospect. But you know, simple file based agentic workflows are extremely powerful. You can see we're generating looks like we have 10 images from our CAT enhanced text file and there it is. So there's all of our images that we generated and all the respective prompts that we use to generate them, right? So you can see we have, you know, a variety of prompts here. Black cat, green eyes, sitting on chair. Rule of thirds, strong sunlight, rainy window, sits on chair. And then we can just look at the exact images generated, right? Cat sitting on chair, sunlight coming through the window, rainy day, there's another one sitting, there's a group of cats, cat with the yarn ball, sleeping cat by the window, Classic stretching cat, cat in some majestic place. Okay, so you get the idea, right? Our agent workflow can now solve this problem repeatedly. Okay, we have cloud code, the top agent here, there's something going on with Gemini cli, right? It's not really built for this yet. Same with the codec cli. I'm waiting for these model providers to really build out everything that you need for a great agent encoding tool that's probably a dedicated video on its own. Just talking about all the essential features that every agent should have. Obviously it's going to be a lot of what cloud code has. You know, you can see here. Theoretically you can wire up any agent to operate on whatever workflow you have end to end. Okay, this is the agentic drop zone. Input files drop into specific directories, kicking off specific agents, running specific models on specific prompts that generate outcomes that you're looking for. And the key here

is repeat workflows, right? If you're working with media, you're going to have to generate hundreds and thousands of files, you're going to need to edit them. And if you're working with training data, you're going to need to build, tweak and modify training data. So why not set up a clean repeat file based workflow to do this for you, right? The workflow that really inspired this agentic drop zone architecture was really this morning debrief. Every morning I have this process where I just record my thoughts, my plan for the day, my plan for the week, goals I want to hit, so on and so forth, right? And I just record it and then I drag and drop it, I get it onto my device, I transcribe it and then I have an agent review it, add notes, add some interesting ideas. You know, you saw that get spit out right here. If we enter morning debrief, debrief output, it generated this great debrief, drag it here. You know, we have this great write up here. For this example, I gave the audio of a previous YouTube video. But you get the idea, right? Every morning I'm working with agents, gathering, collecting, prioritizing my ideas, getting feedback on them, right? Key ideas, extensions, leading questions. There are many, many, pretty much infinite workflows to build out here. This architecture has been very valuable to me, so I wanted to share it here with you. Let's briefly talk about how this works at a code level and more specifically, I don't want to walk through this with you. What I want to do here is highlight the importance of having programmable agents. As mentioned, the chat interface is just the entry point, right? It's the, the first, most fundamental way to interact with intelligence, with LLMs, with AI agents, with agentic workflows. But it's just the beginning, right? Then drop zones. Let us reuse a popular, extremely common interface to interact with our AI, to interact with our compute. It's the simple file based system, your file system, okay, we have files, we have directories. Why not reuse tried and true technology to deliver new value with generative AI? The key here is no matter what workflow you're setting up, you need programmatic agents, all right? And Claude code is absolutely leading the way for programmatic agents. They have two SDKs. They have a TypeScript SDK and a Python SDK. We're using, of course, the Python SDK. Basically it lets you program with cloud code. We had a big video on this a couple months ago where we covered this main idea, but we have built this into our system, right? So if we just copy this, search for this, you can see we have this built into the system, right? Prompt, cloud code. All right? And if we just collapse everything inside of our agents, you can build this out for any agent. Gemini Codex is still a work in progress right now. Cloud code is the primary completed implementation. Gemini and Codex just don't have SDKs built in yet, so you need to go right to the sub process, which takes a little bit more effort to work with with cloud code. You know, you can see here we're building our prompt and this method is super simple, right? We're taking whatever promptly pass in, let's say our image generation prompt. And we're just replacing this with the file path that we dropped in, right? So that our agent can read the specific file. So that's all the build prompt is doing. And then we're taking that full prompt. We're making sure that everything is set up properly and then we're kicking off the cloud code SDK and continuously get

results out of it. Whenever there's a message, right? Whenever cloud code is responding to us and reporting some progress, it just spits that out. We're using rich logging here for a great terminal based interface. And that's basically the whole process, right? We're taking the prompt, we're setting up some configuration and then we're executing it programmatically. Okay? This is huge. Agentic workflows are powerful because it helps us get out of this human in the loop mindset, right? We're so stuck in this chat interface across all generative AI domains, right? We're prompting back and forth and this is something that I say all the time now on the channel. Do you want to be stepping out of this realm? Think about the repeat workflows that you can now automate with agents, okay? And then find yourself a great agent SDK. Obviously on the channel, we're big fans of plot code because they are the leaders in the space. But I am preparing and very ready for any agent to pop up. Right? You can see here we have that Gemini Echo drop zone. Still having some problems working through these other agents. But I think it's all about the direction that's important here. Right? A big fan of cloud code, but you want to prevent lock in. It's too early to pick a winner. With that being said, I'm using cloud code 24 7. They have the best support for the real agentic engineering work that we're all going to be doing in the very near future, if you're on the edge, you're already doing a lot of this agentic work. Let's get out of the simple chat interface, right? Even right here with cloud code, right? This is a chat interface. Let's push beyond this. Let's build out agentic workflows, right? They're called agents for a reason. They're capable of agency. So let's lean into the autonomy and build workflows that operate end to end on our behalf. This is a massive theme of the Phase 2 agentic coding course I'm building right now. It's the successor of principled AI coding. This back and forth in the loop prompting is just, just the beginning of what's possible. All right? So make sure you subscribe so you don't miss the release. It's going to be one of the most valuable things I've ever built. It's shaping up to be an incredible resource. A lot of engineers are going to get massive value out of it. So make sure you subscribe. Make sure you're plugged into the channel so you don't miss the launch. We want to be getting out the loop more and more. We are engineers, so we can build out and leverage this technology better than anyone. Now, when you're building, right, when you're operating, of course, there's nothing wrong with hopping in to a cloud code and sends to a Gemini CLI instance to a Codex instance, whatever. But, you know, more and more, we should be automating work with these agentic workflows. And the agentic drop zone is one way you can do this end to end. It is so funny that prompt engineering is the most important skill any engineer can have now. So whenever you see a prompt, spend some extra time and really dig into it. Ask questions like, why is this prompt structured like this? Look at interesting prompting techniques like we have here, right? We have variables that we're reusing throughout the workflow. We're embedding some XML so that we can specify specific sections and so that we can template out specific variables. So the rest of this single file agent drop zone is actually fairly easy to understand. You can imagine what happens, right,

if we go to main. All we're doing here checking environment variables and we're passing in our configuration file and then we're hitting run. The agent architecture is super simple. If we open up the readme, you can see exactly how this works. All right? Files dropped. We're using Watchdog to detect events. If we match a pattern, we load the Prompt template. If we don't, we ignore replace the file path, we select our agent and then we run whatever work we need to do with the agent, stream the response and we display it. So that's exactly how this works. And the key features here of the agentic drop zone is this, right? Single simple file script, triggerable drop zones. It has the framework for running any agent. You can run them in parallel and you can run arbitrary agentic workflows. I've been playing with this quite a bit, thinking about all the workflows that I do on a daily basis with files. I recommend you do the same. All it takes is a single input file and then you can encode repeat solutions. Just dropping files into directories and we're getting work done, right? Super simple, really powerful idea. The chat UI is the first, simplest and lowest hanging fruit. Agentic drop zones or simply put, you know, these reactive directories let you automate file input based workflows. We're going to have more agents available to us, we're going to have more powerful models. I can guarantee you right now, the models, the agents, they are no longer the bottleneck, okay? It's now a complete skill issue. You can do incredible things with this technology if you're plugged into the right information channels, of course, like this channel. We talk about big ideas like this every single week. Make sure you're plugged in so you don't miss what you can do. Okay? This is yet another big idea I'm sharing here on the channel with you. As you saw, I have a workflow that takes my morning debrief, it transcribes it, it adds additional thoughts formats, it, it prioritizes things for me. And now this is just available for me to drag and drop every single day, over and over and over. Okay? So think about how that stacks up, right? Think about as you build up these agent workflows. Think about the lead. Think about the advantage you gain by using the right compute in the right agent architecture, like the agentic drop zone. This code base is of course going to be available to you. Link in the description on the channel. We're going to be working to step outside the chat box. Think about all the repeat engineering work you do and think about how you can automate it with an agentic workflow. No matter what you do, stay focused and keep building.